Project Title: **Temperature-Regulated CPAP system for Sleep Apnea management**

BME 3200: Biomedical Systems Design Project

A project submitted to the Department of Biomedical Engineering in Third Year Second Semester



Supervised by	**Submitted by**
Dr. Md. Bashir Uddin Professor Department of Biomedical Engineering	**Name**: Shaira Afra Redhila **Roll**: 2015008 Department of Biomedical Engineering Khulna University of Engineering & Technology Khulna-9203, Bangladesh

Abstract

This project focuses on developing a cost-effective thermistor-based system to detect breathing abnormalities and provide automated air pressure assistance. By monitoring temperature variations during breathing, the system identifies issues when the temperature drops below a set threshold and activates a motor to deliver air pressure for a fixed duration.This project emphasizes affordability, accessibility, and effective treatment for individuals in resource-constrained environments. Additionally, graphical visualization of temperature fluctuations during breathing cycles is introduced to enhance diagnostic accuracy and user interaction.

Table of Contents

Title Page	Page 1
Abstract	Page 2
Table of Contents	Page 3-4
List of Tables	Page 4
List of Figures	Page 5-6

Index

Chapters	Contents	Page no.
Chapter 1	1.1 Sleep Apnea	7
	1.2 Types of CPAP Machines	8
	1.3 Working Principle of CPAP Machines	10
	1.4 Components of a CPAP Machine	11
	1.5 Pros & Cons	12
	1.6 Advancements in CPAP Technology	13
	1.7 Future Trends	14
Chapter 2	Motivation and background	15
Chapter 3	3.1 System design	16
	3.2 Theoretical background & stepwise implementation	17
Chapter 4	4.1. Hardware Implementation	20
	4.2. Software Implementation	22

	4.3 Details of The Components	25
Chapter 5	5.1. Results 5.2. Result analysis 5.3. Challenges and Future works	31 32 34
Chapter 6	Discussion	35
Chapter 7	Conclusion	36
	References	37
	Code	38

List of Tables

Table no.	Table title	Page no.
1	Pros and cons of using a CPAP machine	12
2	List of Hardware Components	20
3	Future improvements scopes of my project	34

List of Figures

Figure no.	Description	Page no.
1	CPAP therapy breaks the cycle of OSA	8
2	CPAP therapy: flow generator, hose, orinasal mask	9
3	Typical full face CPAP mask	11
4	CPAP Nasal Pillows mask	12
5	Prisma Smart	13
6	ResMed AirMini Travel CPAP	13
7	Block diagram of methodology of thermistor-Based Low-Cost CPAP System	16
8	12V battery to power a 12V air pump	17
9	Thermistor embedded into the mask	18
10	Motor driver LM298N connection to Arduino	19
11	OLED display	19
12	Implemented final circuit	19
13	DC 12V Air Pump	25

14	Thermistor	26
15	Medical grade oxygen Mask	26
16	L298 Motor Driver	27
17	Switch	27
18	LM2596 DC-DC Buck Converter	28
19	OLED Display	29
20	12V Battery	29
21	Arduino UNO	30
22	Final circuit visualization	31
23	Data visualization in serial monitor and serial plotter using Arduino IDE	31

CHAPTER 1

Objectives

- Develop a thermistor-based system to detect temperature changes associated with breathing.
- Integrate the detection mechanism with a CPAP machine to automate airflow delivery during apneic events.
- Provide a real-time graphical representation of breathing patterns for monitoring and diagnostic purposes.
- Ensure affordability and reliability of the overall system to make it accessible for widespread use.

Introduction

Snoring is the sound caused by the vibration of throat tissues during sleep when airflow is partially obstructed. It happens due to relaxed throat muscles, a narrowed airway, or blockages in the nasal passages. Common causes include obesity, alcohol consumption, nasal congestion, or conditions like sleep apnea.

1.1 Sleep apnea

Sleep apnea is a chronic condition characterized by repeated interruptions in breathing during sleep. These interruptions, called apneas, can last from a few seconds to minutes, disrupting the sleep cycle and reducing oxygen supply to the body. Untreated sleep apnea is associated with numerous health risks, including cardiovascular diseases, hypertension, type 2 diabetes, and cognitive impairments.Sleep apnea is broadly categorized into three types:

Obstructive Sleep Apnea (OSA): Obstructive Sleep Apnea (OSA) is a sleep disorder where the airway becomes partially or fully blocked during sleep, leading to breathing interruptions. It is caused by relaxed throat muscles or excess tissue blocking the airway and is associated with risk factors like obesity, smoking, or age.

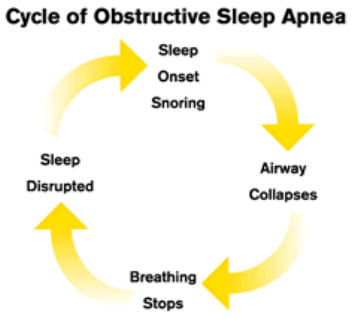


Figure 1: CPAP therapy breaks the cycle of OSA

Other types of sleep apnea include **Central Sleep Apnea (CSA)**, caused by the brain's failure to signal breathing muscles, and **Complex Sleep Apnea**, a combination of OSA and CSA. These conditions primarily occur during sleep due to natural muscle relaxation, gravity pulling tissues backward, reduced breathing effort, and increased airway resistance. If untreated, sleep apnea can lead to serious health issues, including heart disease and high blood pressure.

1.2 Continuous Positive Airway Pressure (CPAP)

CPAP machines are the most commonly prescribed treatment for obstructive sleep apnea. A CPAP machine delivers a steady stream of pressurized air through a mask to keep the airway open during sleep, preventing apneas and improving sleep quality. Modern CPAP machines are equipped with advanced features, including adjustable pressure settings, humidifiers, and data tracking for compliance monitoring.

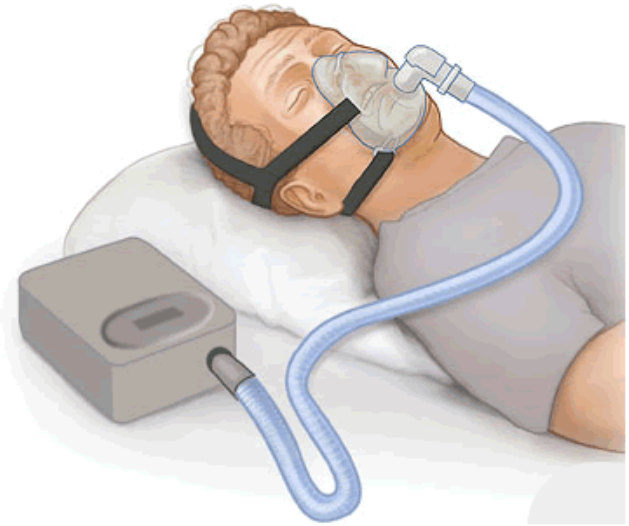


Figure 2: CPAP therapy: flow generator, hose, orinasal mask

Types of CPAP Machines

- **Continuous Positive Airway Pressure (CPAP)**

  CPAP machines deliver a continuous and fixed stream of pressurized air through a mask to maintain airway patency. Initially developed for home treatment of sleep apnea, CPAP is now also widely used in intensive care units. The continuous air pressure prevents airway collapse, enabling normal breathing patterns during sleep. CPAP therapy is highly effective in improving sleep quality, reducing snoring, and enhancing overall quality of life. Typical pressure settings range from 4 to 20 cmH2O, prescribed after a sleep study. However, some patients find the fixed pressure uncomfortable, which can impact adherence to therapy. Ongoing patient education and support are critical to achieving long-term benefits.

- **Automatic Positive Airway Pressure (APAP)**

  APAP devices adjust air pressure dynamically on a breath-by-breath basis, delivering the minimum pressure needed to keep the airway open. They measure resistance in the airway caused by snoring or apnea events and titrate the pressure accordingly. This makes APAP more comfortable compared to CPAP, as it avoids unnecessarily high pressure

when not needed. By personalizing therapy, APAP devices ensure effective treatment while enhancing patient comfort.

- **Bi-level Positive Airway Pressure (BPAP)**

  BPAP devices provide two distinct pressure levels: a higher inspiratory positive airway pressure (IPAP) during inhalation and a lower expiratory positive airway pressure (EPAP) during exhalation. This dual-pressure system makes breathing more natural and comfortable, especially for patients with severe conditions like central sleep apnea or neuromuscular disorders. BPAP devices can operate in three modes: spontaneous (triggered by the patient's breathing), timed (with a fixed breathing rate), and spontaneous/timed (combining spontaneous triggers with a backup rate). This flexibility makes BPAP suitable for a wider range of patients compared to CPAP or APAP.

- **Expiratory Positive Airway Pressure (EPAP)**

  EPAP devices are specifically designed to manage exhalation in patients with OSA or snoring issues. These devices use small valves placed on the nostrils, allowing air to be inhaled but not exhaled freely. This resistance creates back pressure that keeps the airway open during exhalation. Additionally, the resistance may lead to a slight buildup of CO2, which can increase the respiratory drive. EPAP devices are simple and non-invasive, making them an attractive option for certain patients, although they are less commonly used than CPAP or BPAP devices.

1.3 Working Principle of CPAP Machines

Positive airway pressure (PAP) devices work by maintaining the patency of the upper airway, ensuring unobstructed breathing. They do so by applying air pressure, which prevents the airway from collapsing, particularly during sleep. In obstructive sleep apnea (OSA), the muscles of the upper airway relax, leading to narrowing and reduced oxygen levels in the blood. PAP devices deliver pressurized air to splint the airway open, reducing episodes of apnea and hypopnea. It is

important to note that the effectiveness lies in the applied air pressure rather than the movement of air itself.

1.4 Components of a CPAP Machine

A CPAP system typically consists of the following components:

- **CPAP Machine (Main Unit)**: The core component, responsible for generating the air pressure and maintaining a continuous flow.
  - **Motor**: Powers the device and produces air pressure.
  - **Humidifier**: Moistens the air to prevent dryness and irritation in the airways.
  - **Air Filter**: Removes dust, allergens, and other particles from the air.
- **CPAP Mask**: Worn over the nose or mouth to deliver air into the airway.
  - **Nasal Masks**: Cover the nose and are often preferred for people who breathe through their nose.
  - **Full Face Masks**: Cover both the nose and mouth, used by people who breathe through both or only their mouth.



Figure 3: Typical full face CPAP mask

  - **Nasal Pillows**: Insert into the nostrils, offering a more minimalistic design.



Figure 4: CPAP Nasal Pillows mask

- **Hose**: A flexible tube that connects the CPAP machine to the mask. It carries the air to the user..

1.5 Pros & Cons

Table no.1: Pros and cons of using a CPAP machine

Pros	Cons
• improves your ability to sleep without waking up • lowers your risk of having a heart attack, stroke, or other cardiovascular event • helps lower your blood pressure • reduces daytime sleepiness • may reduce blood glucose and cholesterol levels	• discomfort and difficulty falling asleep, especially in the beginning • a feeling of claustrophobia or anxiety • nasal congestion • dry mouth • nosebleeds • skin irritation or sores where the mask touches the face • a feeling of being bloated with air

1.6 Advancements in CPAP Technology

- **Smart CPAP Machines**: These devices are equipped with Bluetooth technology to track sleep data and share it with the user's smartphone or healthcare provider for remote monitoring and adjustments.



Figure 5: Prisma Smart

- **Adaptive Pressure Technology**: Modern CPAPs can adjust air pressure dynamically based on real-time data, enhancing comfort and compliance.
- **Travel CPAP Machines**: Smaller, portable versions are available for users who need to travel frequently, offering more convenience without sacrificing functionality.



Figure 6: ResMed AirMini Travel CPAP

- **Integrated Oxygen Therapy**: Some CPAP machines now include the option to administer supplemental oxygen for patients with low blood oxygen levels.

1.7 Future Trends

- **Miniaturization**: Smaller and lighter CPAP machines with improved comfort and portability are being developed to make therapy more convenient for users.
- **Wearable Technology**: Integration of CPAP therapy with wearables like smartwatches or other health devices may offer real-time monitoring and data analysis.
- **Personalized Pressure Settings**: Emerging technologies may allow for more personalized pressure settings based on the user's sleep cycle and breathing patterns, offering a more tailored approach to treatment.

While CPAP machines are considered the gold standard for managing obstructive sleep apnea, their high cost often limits accessibility, particularly in low-resource settings. This project aims to address this gap by developing a low-cost CPAP system that uses a thermistor to detect apneas based on temperature fluctuations in exhaled and inhaled air.

Chapter 2

Motivation

Sleep apnea affects millions of individuals worldwide, with a significant portion of the population remaining undiagnosed and untreated. The high cost of conventional CPAP machines and diagnostic tools creates a barrier for individuals in low-resource settings. The inspiration for this project came from both personal and academic reasons. My supervisor, who has extensive experience in sleep apnea research, provided invaluable insights that shaped the foundation of this work. Additionally, my mother's struggle with snoring and breathing difficulties motivated me to explore a solution that could address these issues effectively. Furthermore, I was inspired by OxyJet, a low-cost CPAP system developed by BUET, which highlighted the potential of creating affordable solutions. However, this project diverges from OxyJet, which aims to address financial constraints by developing a low-cost CPAP system that uses a thermistor to detect apneas based on temperature fluctuations in exhaled and inhaled air.

Background Study

Research into sleep apnea has highlighted its widespread prevalence and serious health implications. Studies from the American Academy of Sleep Medicine and the World Health Organization (WHO) emphasize the urgent need for early detection and intervention. Existing CPAP machines are effective but often inaccessible due to their cost. Recent advancements in sensor technologies, particularly thermistors, have shown promise in respiratory monitoring due to their sensitivity and affordability.

Thermistors are widely used in biomedical applications for their ability to detect minute temperature variations. They have been successfully implemented in devices that monitor breathing patterns, making them an ideal choice for detecting apneas based on temperature fluctuations. Literature also supports the integration of microcontrollers and data visualization techniques as effective methods for enhancing user interaction and diagnostic accuracy.

Chapter 3

Methodology

3.1 System design

Here the system design and the device's working principle are described. The main units are connected as shown in the block diagram (Figure 1). This block diagram basically helps us to understand how all the equipment is connected to each other.

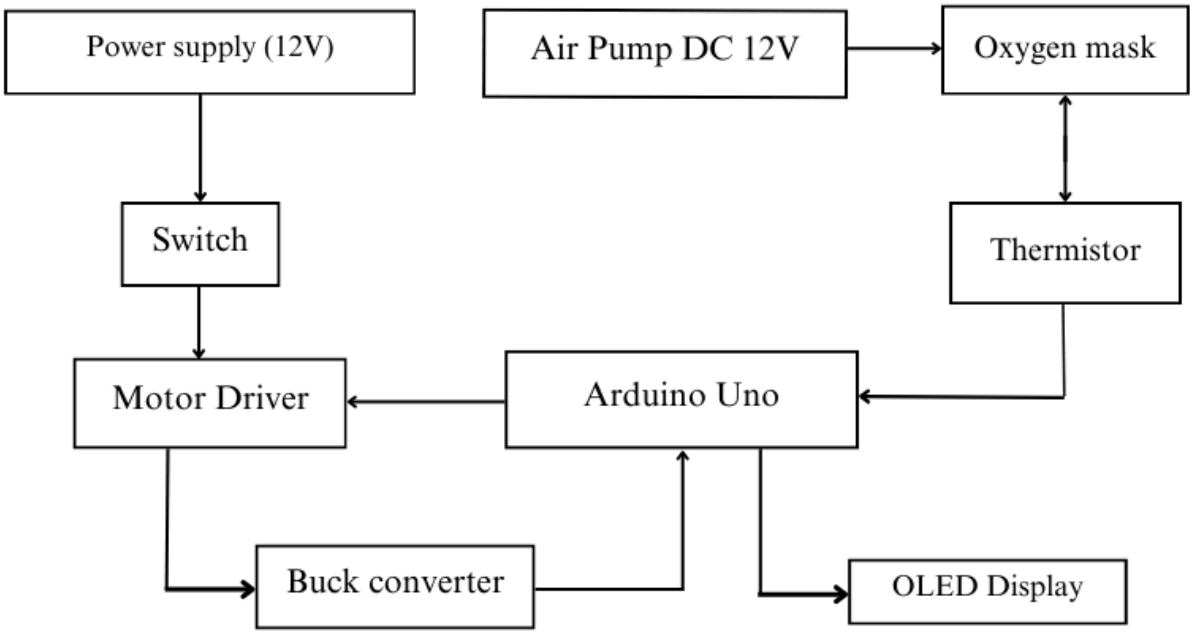


Figure 7: Block diagram of methodology of thermistor-Based Low-Cost CPAP System

3.2 Theoretical background & stepwise implementation

Power Supply Design

The project methodology involved systematically integrating and programming the components to create a low-cost CPAP machine for sleep apnea management. The design incorporates a 12V battery to power a 12V air pump (previously specified in earlier documentation), ensuring continuous airway pressure delivery. To accommodate the voltage requirements of other components, a buck converter was used to step down the 12V supply to 5V. This 5V output powers the LM298N motor driver, the Arduino Uno microcontroller, and the OLED display, all of which operate efficiently at this lower voltage.

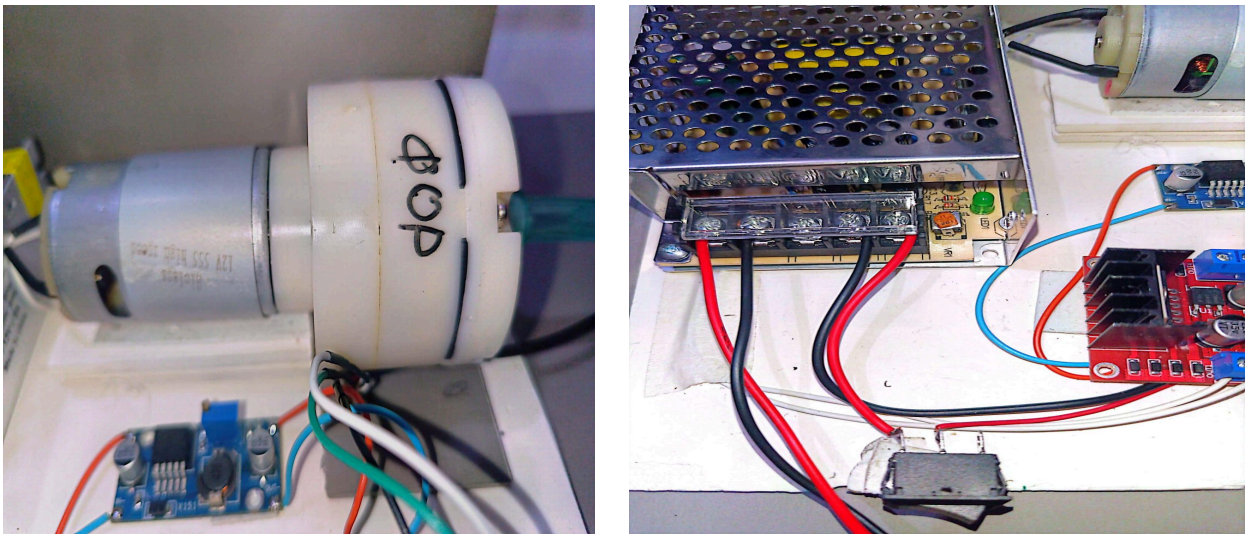


Figure 8 : 12V battery to power a 12V air pump

Breathing Monitoring and Feedback Loop

A thermistor was embedded into the mask to monitor temperature variations during the user's breathing cycle. The thermistor provides real-time data on temperature changes associated with inhalation and exhalation. The Arduino Uno processes this data and compares it to a predefined temperature threshold. When the temperature exceeds the threshold, the system maintains a steady low-pressure airway flow.

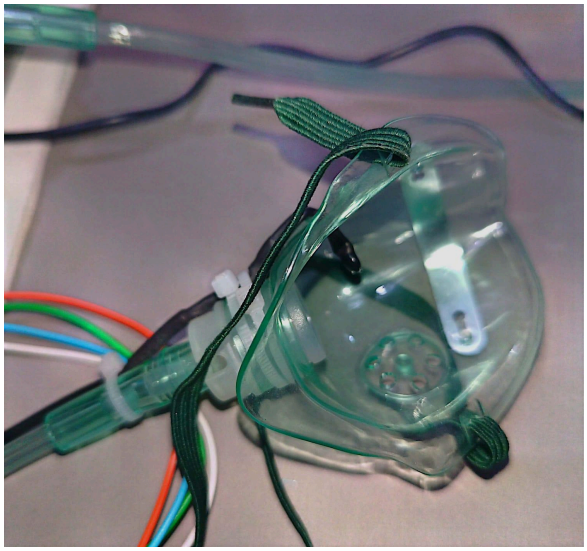


Figure 9 : Thermistor embedded into the mask

Conversely, when the temperature falls below the threshold, the system increases airway pressure for a duration of 15 seconds, as programmed in the Arduino. This temporary increase aims to stabilize the airway. After 15 seconds, the system re-evaluates the temperature data from the thermistor. If the temperature remains below the threshold, the elevated pressure loop is re-initiated. However, if the temperature exceeds the threshold, the system resumes the baseline low-pressure airflow.

Control Mechanism

The programming logic was adjusted to ensure seamless transitions between pressure levels and to establish a continuous feedback loop based on the thermistor's readings. The motor driver LM298N was employed to control the air pump's operation, effectively managing the pressure changes.

User Interface

Additionally, the OLED display provides a user interface for monitoring system parameters and real-time status.

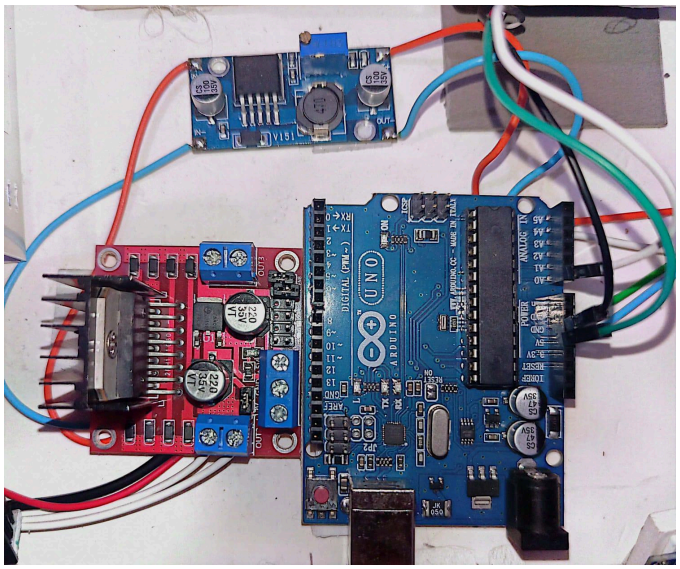


Figure 10: Motor driver LM298N connection to Arduino

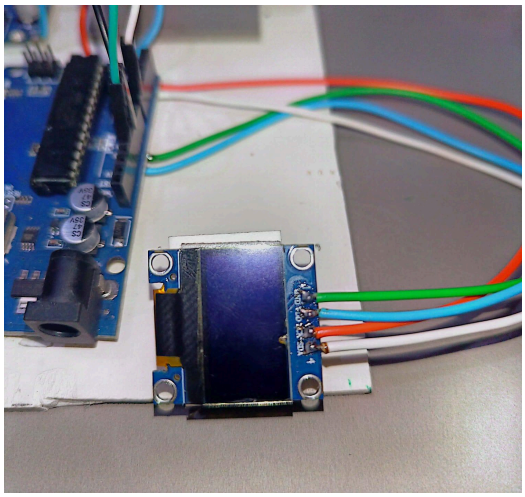


Figure 11: OLED display

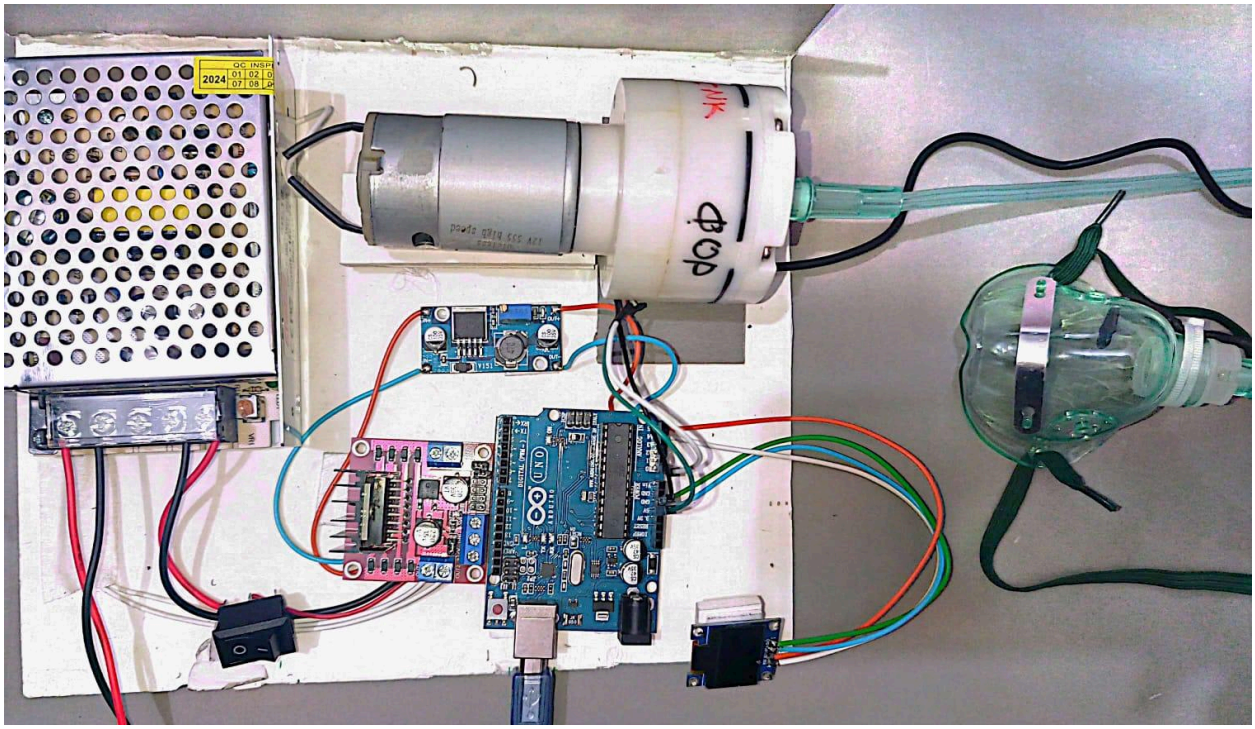


Figure 12: Implemented final circuit

Chapter 4

Components

4.1. Hardware Implementation

Table no 2. List of Hardware Components

Serial no.	Component	Description	Rating	Cost(BDT)
1.	Air Pump DC 12V 555 Diaphragm Flow Vacuum Air Pump Compressor	Provides adjustable airflow pressure for simulating CPAP functionality.	Voltage: 12V, Pressure: ~200 mmHg	640
2.	L298 H-Bridge Dual Motor Driver	Controls the air pump motor with PWM for varying pressure levels	Voltage: 5V-35V, Current: 2A per channel	180
3.	Arduino Uno	Microcontroller board used to control the system and process sensor data.	Voltage: 5V, Input Voltage: 7-12V	950
4.	LM2596 DC-DC Buck Converter	Regulates voltage to ensure stable operation of components.	Input Voltage: 4V-40V, Output: 1.23V-37V	80

5.	OLED Display	Displays status messages like normal respiration or respiratory issues.	Voltage: 3.3V-5V, Size: 0.96 inches	310
6.	12V Battery	Provides power to the air pump and the system.		500
7.	Switch	Manually turns the system on or off.		5
8.	Jumper wires	For circuit connection		40
9.	Oxygen mask	Delivers air from the pump to the user.		60
10.	Thermistor	Senses temperature changes for respiration monitoring.	Resistance: 10kΩ at 25°C Type: NTC	185
				Total=2950

4.2. Software Implementation

The software implementation for this project involves programming the Arduino Uno microcontroller using the **Arduino IDE.** The code is written in C++ and is designed to monitor, control, and display the status of the respiration system.

Operational breakdown of code

1. Library Inclusions and Pin Definitions

The code begins with the inclusion of necessary libraries for handling sensors, OLED displays, and other hardware components:

- **DHT.h**: Library for DHT sensors (though unused in this code).
- **LiquidCrystal.h**: For LCD control (not used in the current code).
- **Adafruit_Fingerprint**: For handling fingerprint sensors (not used).
- **Adafruit_SSD1306**: For managing the OLED display.
- **Thermistor pin (A0)**: Analog input for reading temperature.
- **Motor control pins**: in1, in2, y1, y2, EN1 to control a motor (air pump) via an L298 motor driver.

2. Global Variables and Constants

- **Temperature Calculation Constants:** R1, c1, c2, and c3 are used for the Steinhart-Hart equation to convert the thermistor resistance to temperature.
- **Motor Speed (Speed):** Initial PWM value for motor control is set to 90.

3. OLED Initialization

The OLED display (**Adafruit_SSD1306**) is initialized with:

- **Screen size**: 128×64 pixels.
- **Startup message**: Displays "Starting" on the OLED during setup.

4. setup() Function

- Initializes Serial communication for debugging (115200 and 9600 baud rates).
- Initializes the OLED display and outputs a "Starting" message.
- Configures all required pins for OUTPUT or INPUT modes.
- Sets up the motor driver with initial states (LOW for y1 and y2, PWM for EN1).

5. loop() Function

This is where the core functionality runs, involving temperature monitoring and controlling the air pump based on thresholds:

(a) Read and Calculate Temperature

1. **Thermistor reading**:
    - Vo = analogRead(ThermistorPin) reads the analog voltage.
    - Converts it to resistance R2 and uses the Steinhart-Hart equation to calculate the temperature in Kelvin (T), Celsius (Tc), and Fahrenheit (Tf).
2. **Display Temperature**:
    - Outputs temperature on the OLED display and the Serial Monitor.

(b) Control Logic Based on Temperature

1. **If Tf >= threshold (Normal)**:
    - Turns ON y1 (motor control pin) and sets a moderate motor speed (analogWrite(EN1, 100)).
    - Displays "Normal" status on the OLED.

2. **If Tf < threshold (Hampered):**
    - Increases motor speed for 15 seconds to compensate for the low temperature.
    - Displays "Hampered" status on the OLED.
    - Recalculates temperature every 500ms during this period and adjusts motor speed accordingly.

6. Display Updates

The OLED display is dynamically updated in both states:

- **Normal Mode**:
    - Displays "Temp(F)" and "Normal".
- **Hampered Mode**:
    - Displays "Temp(F)" and "Hampered".

7. Motor and Pump Control

The motor driver (L298) controls an air pump:

- **y1 and y2:** Set the motor direction.
- **EN1:** Controls motor speed using PWM (Pulse Width Modulation).
- Motor speed increases when the temperature is below the threshold.

8. Serial Debugging

Throughout the code, the Serial Monitor outputs:

- Current temperature readings in Fahrenheit.
- Debugging messages for troubleshooting.

4.3 Details of The Components

DC 12V Air Pump 555 Diaphragm High Flow Vacuum Air Pump Compressor for Oxygen Gas Air Pump

The air pump is the primary component responsible for generating a continuous flow of pressurized air. Operated at 12V, this diaphragm-type pump is compact and efficient, capable of producing pressures around 200 mmHg. The pump operates on the **diaphragm mechanism**, powered by a DC motor. When the motor rotates, it drives an eccentric cam connected to a flexible diaphragm. The diaphragm moves up and down, creating pressure changes inside the pump chamber.

- **Compression Stroke**: The diaphragm's downward motion compresses the air, forcing it out through the outlet valve.
- **Suction Stroke**: The diaphragm's upward motion creates a vacuum, drawing air or gas into the chamber through the inlet valve.

An **eccentric cam** is a mechanical component that converts the rotational motion of a motor into linear (up-and-down) motion. In the context of the diaphragm pump, the cam is mounted slightly off-center (eccentric) on the motor's rotating shaft.As the motor shaft rotates, the eccentric cam moves in a circular path, causing a connected rod or arm to push and pull the diaphragm in a cyclic motion.This cyclic movement creates alternating suction and compression strokes in the pump chamber, enabling the air or gas to be moved effectively.

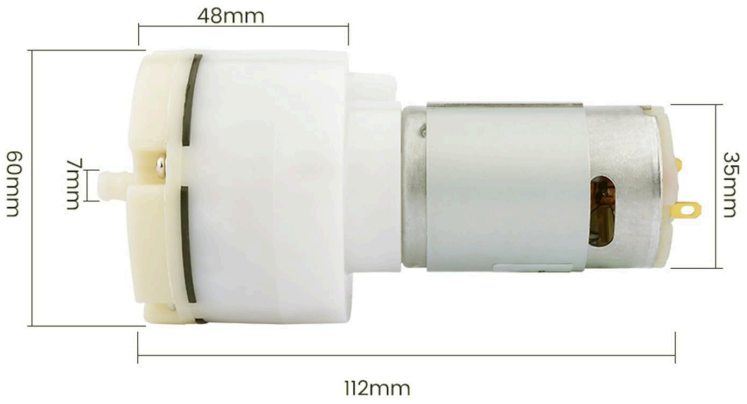


Figure 13: DC 12V Air Pump

Thermistor

The thermistor is a temperature sensor used to monitor the user's respiration. Positioned near the user's nasal or oral cavity, it detects temperature changes caused by inhalation and exhalation. This information helps determine if the user's respiration is normal or if there are potential issues. The thermistor used in this project is an NTC (Negative Temperature Coefficient) type, with a resistance of 10kΩ at 25°C. Its sensitivity to temperature variations makes it a suitable choice for respiratory monitoring.



Figure 14: Thermistor

Oxygen Mask

The oxygen mask delivers air from the pump to the user's airway. Made from medical-grade silicone, it ensures safety, comfort, and durability, making it suitable for prolonged use. The mask is designed to distribute pressurized air effectively, ensuring the user receives the intended airflow to maintain normal respiration. Its ergonomic design and soft material provide a snug fit, preventing air leakage and enhancing user comfort.



Figure 15: Medical grade oxygen Mask

L298 H-Bridge Dual Motor Driver

The L298 motor driver serves as the control interface between the Arduino microcontroller and the air pump. It facilitates precise control of the pump's motor speed and direction using Pulse Width Modulation (PWM). This dual-channel driver supports two DC motors and operates within a voltage range of 5V to 35V, with a current capacity of up to 2A per channel. Its robust design allows it to handle the power demands of the air pump while enabling efficient regulation of airflow pressure based on system requirements.

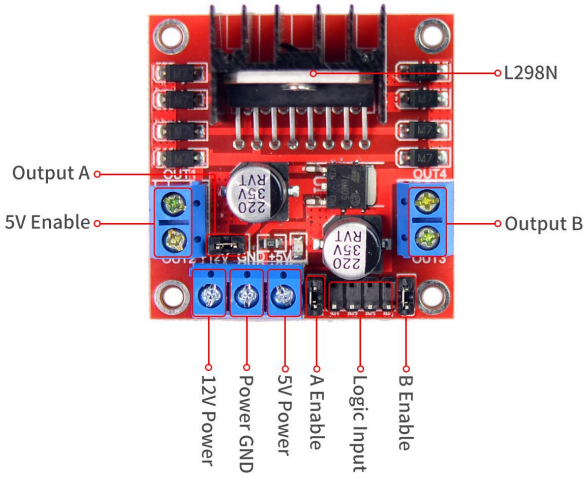


Figure 16 : L298 Motor Driver

Switch

The switch provides a manual on/off control for the system, offering a simple and intuitive way to activate or deactivate the device. It is designed to handle the system's operating voltage (12V) and current requirements, ensuring safe and reliable operation. The inclusion of a switch adds an element of user convenience, enabling quick control of the system.



Figure 17 : Switch

LM2596 DC-DC Buck Converter

Voltage regulation is essential for ensuring the stable operation of the components, and the LM2596 DC-DC buck converter plays a crucial role in this. It steps down the higher input voltage (up to 40V) to the lower levels required by various components like the air pump, Arduino, and OLED display. The converter's output is adjustable between 1.23V and 37V, making it highly versatile. Its high efficiency ensures minimal power loss, contributing to the system's overall reliability and energy efficiency.

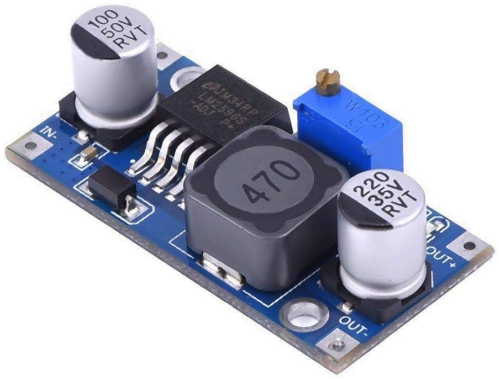


Figure 18 : LM2596 DC-DC Buck Converter

OLED Display

The OLED display is used to provide visual feedback about the system's status. Instead of showing raw temperature readings, the display communicates whether the user's respiration is normal or if there is an issue. Measuring 0.96 inches diagonally, it offers high-contrast visuals that are easy to read, even in low-light conditions. Operating on 3.3V-5V, this compact display integrates seamlessly with the Arduino, enhancing the system's user-friendliness.

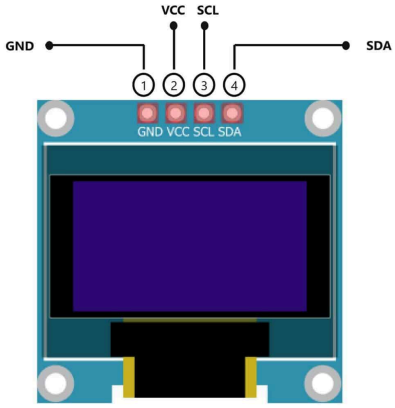


Figure 19 : OLED Display

Battery

The 12V battery powers the entire system, making it portable and convenient for real-world applications. It supplies energy to components such as the air pump, Arduino, and motor driver, ensuring uninterrupted operation. The battery's capacity determines the duration for which the system can function without external power, making it a critical component for maintaining system autonomy during extended use.



Figure 20 : 12V Battery

Arduino Uno

The Arduino Uno acts as the central processing unit of the system, coordinating all operations. It processes inputs from the thermistor, determines the respiratory state, and controls the air pump via the motor driver. Built on the ATmega328P microcontroller, the Arduino offers versatile input/output capabilities, including digital and analog pins, making it ideal for interfacing with sensors, displays, and actuators. Operating at 5V, it provides a reliable platform for real-time monitoring and control in this project.

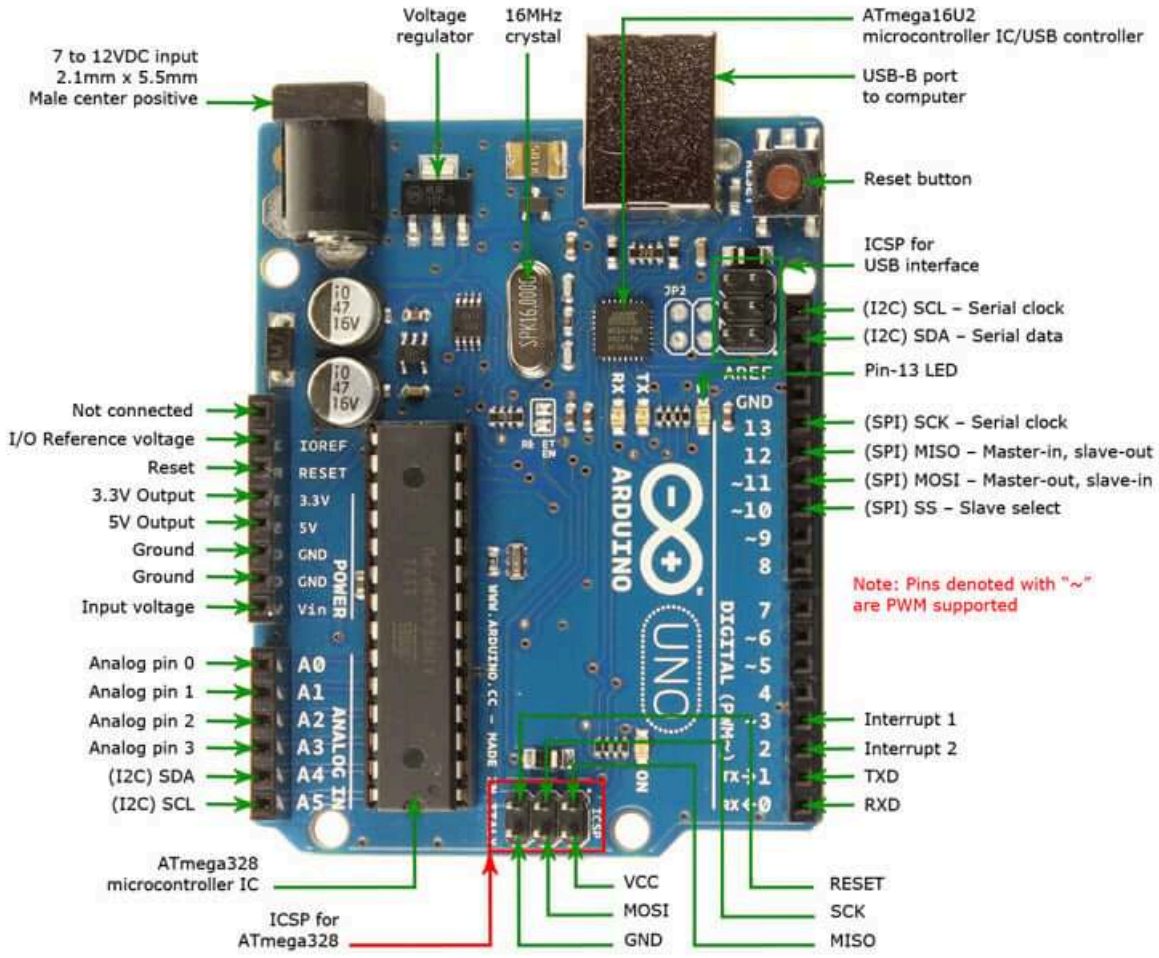


Figure 21: Arduino UNO

Chapter 5

Result & Discussion

5.1 Results

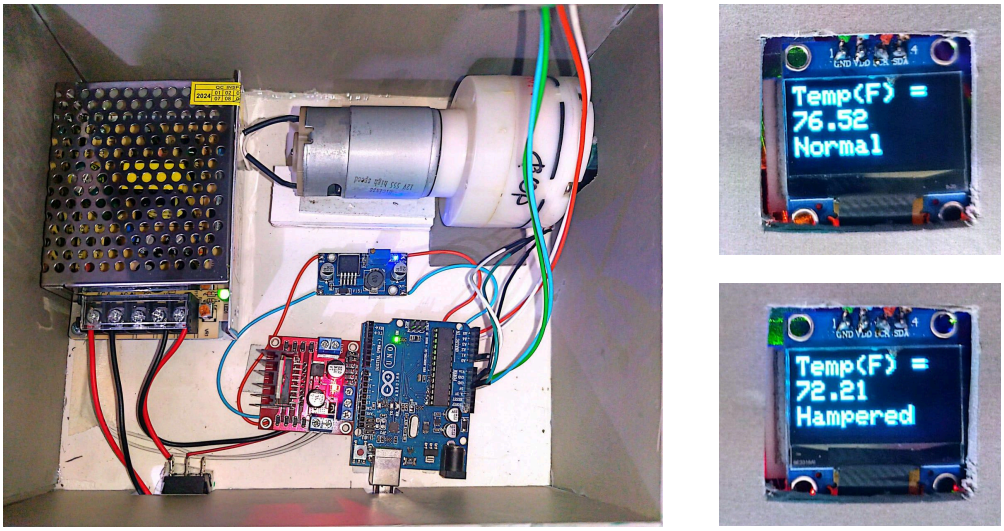


Figure 22: Final circuit visualization

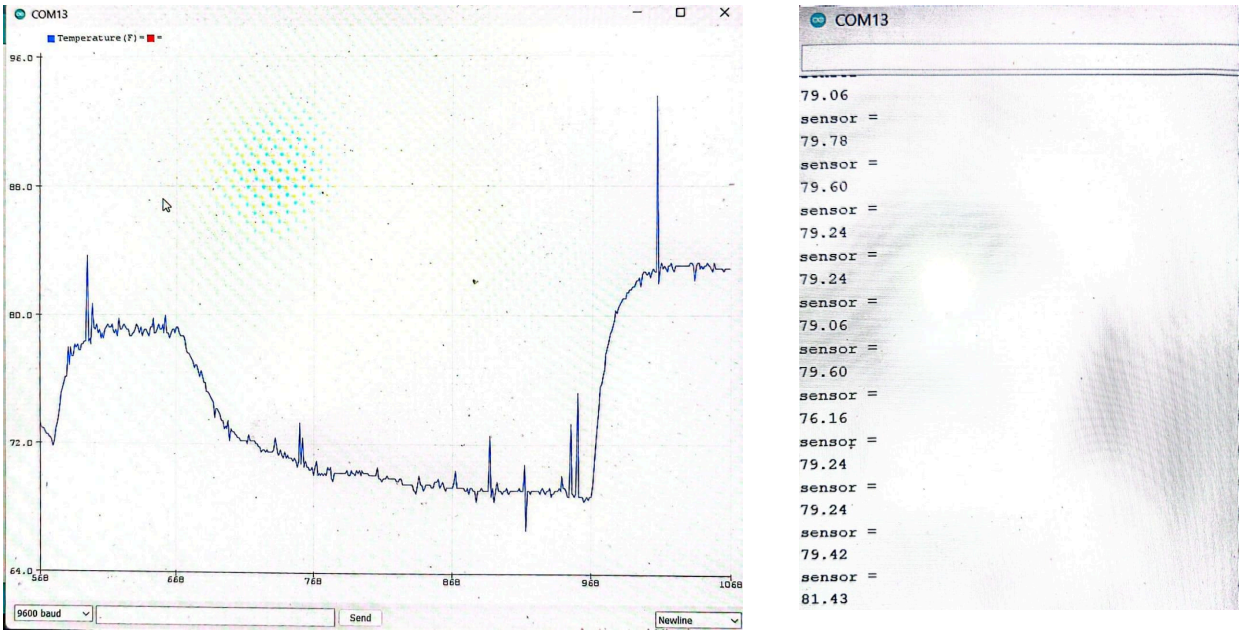
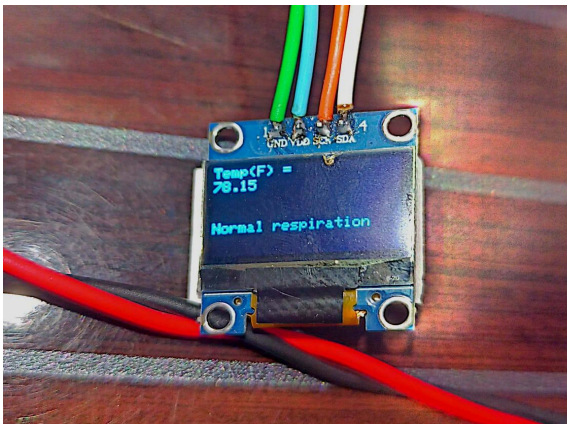
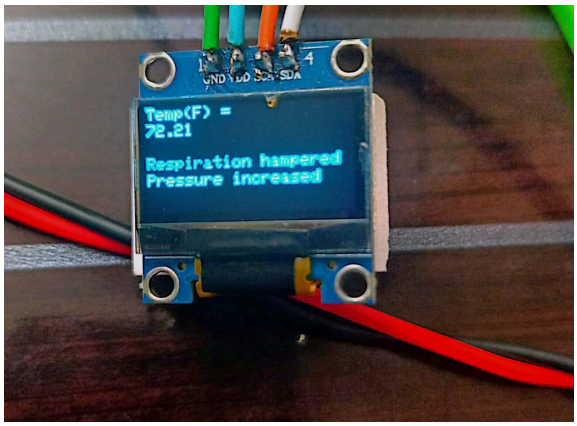
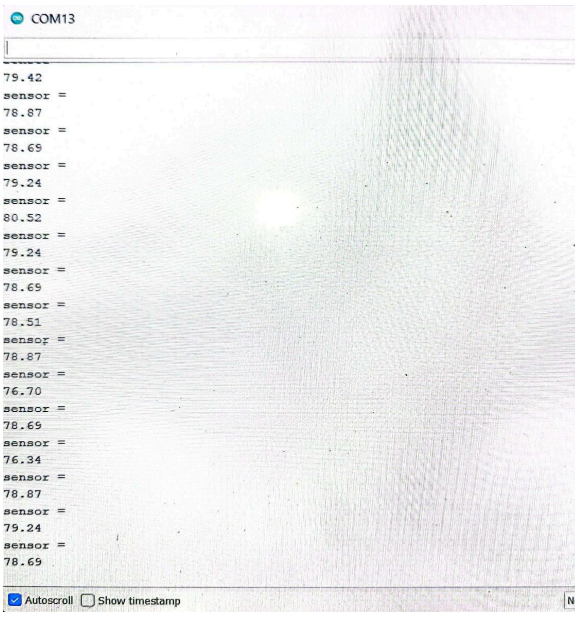
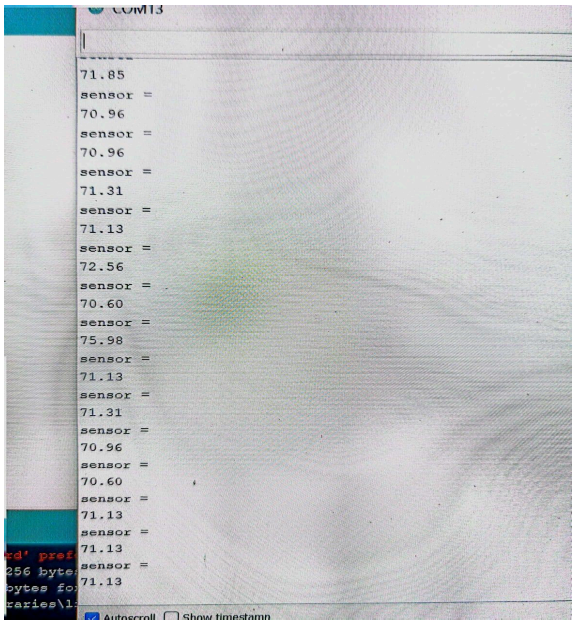


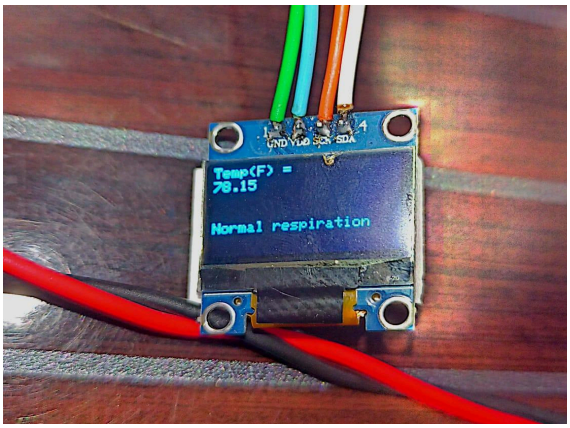
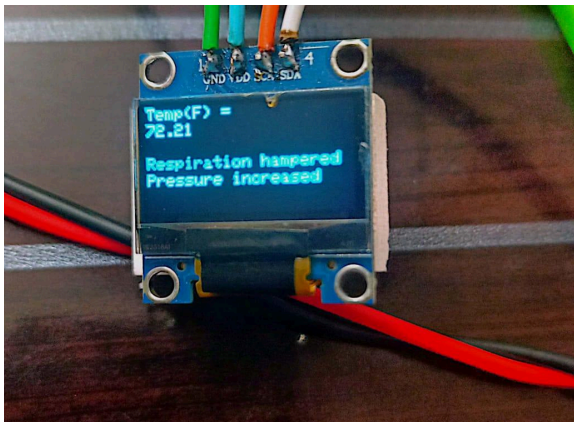
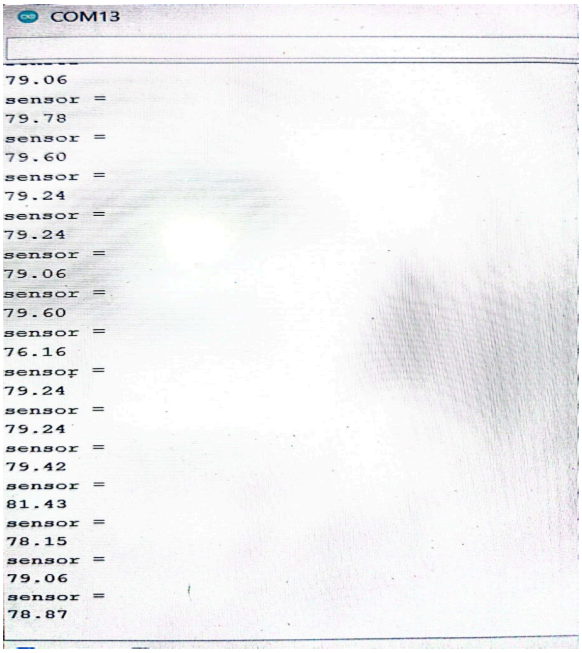
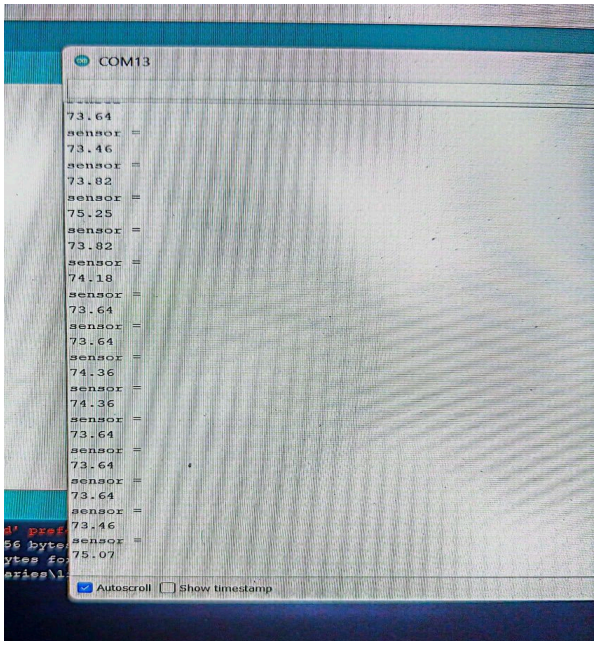
Figure 23: Data visualization in serial monitor and serial plotter using Arduino IDE

5.2 Result Analysis

I. At Room Temperature around 26°C (78.8°F)

During normal respiration	While respiration gets hampered
	
	
Temperature ~76–80°F	Temperature ~70–71°F

II. At Room Temperature around 18°C (64.4°F)

During normal respiration	While respiration gets hampered
	
	
Temperature ~79–81°F	Temperature ~73–75°F

5.3 Challenges and Future works

While the implemented system meets the basic objectives, there were some challenges and trade-offs:

- The lack of an ambient temperature sensor means the threshold is static, which could make the system less effective in changing environmental conditions.
- Without a pressure sensor, the system relies entirely on temperature data, which limits the accuracy and efficiency of its decision-making process.
- Machine learning, which could have improved breathing pattern detection, was not incorporated due to complexity and resource constraints.

Table 3: Future improvements scopes of my project

Improvement	**Details**
Machine Learning	Implement AI algorithms to predict apnea events and improve system response.
Remote Monitoring	Add IoT capabilities for healthcare professionals or users to monitor and control the device remotely.
Enhanced Data Logging	Enable data storage for tracking long-term health trends and user compliance.
User Feedback System	Allow users to provide feedback on comfort and performance for continuous improvements.

Chapter 6

Discussion

Initial Plan and Vision

At the start, my vision for this project was to build a comprehensive and dynamic system to detect and respond to breathing abnormalities. I had planned to use an ambient temperature sensor to implement dynamic thresholding. This would allow the system to adjust its baseline temperature threshold based on the surrounding environmental conditions, making it more adaptive and accurate.

Additionally, I wanted to incorporate a pressure sensor to monitor pressure variations during breathing. This sensor would provide valuable data about airflow and breathing patterns, allowing the system to make more precise decisions about when and how to adjust the motor speed. I even considered using machine learning to analyze breathing patterns for more accurate detection and response. The idea was to create a smart system capable of making efficient and informed decisions based on real-time data.

What I Actually Implemented

Due to time and cost limitations, I had to simplify my approach. Instead of using dynamic thresholding or additional sensors, I relied solely on a thermistor to measure temperature changes. The system uses a fixed threshold to determine whether the temperature is within a normal range or has dropped below the set point, indicating a potential issue.

The motor runs at a low speed under normal conditions and switches to a high speed when the temperature falls below the threshold for a continuous 10-second period. After running at high speed for 15 seconds, the system resumes monitoring to decide whether to continue at high speed or return to low speed. The OLED display provides real-time updates, showing the current temperature and system status, making it easy to monitor the system's performance.

Chapter 7

Conclusion

This project demonstrates a cost-effective and innovative approach to managing obstructive sleep apnea using a thermistor-based mask and a 12V air pump. Despite all the limitations, the project achieved a functional and reliable system. The thermistor-based temperature monitoring works well to detect abnormal conditions, and the motor responds accordingly. The OLED display provides clear feedback, making the system easy to use.

This project taught me how to balance ambitions with practical constraints and make the most of available resources. I also learned about the importance of planning and the challenges of integrating multiple components in a single system. If given more time and resources, I would revisit my original goals and add features like dynamic thresholding, pressure sensors, and machine learning to make the system smarter and more efficient. With further improvements like noise reduction, air filtration, and advanced sensor integration, this project holds the potential to provide a more accessible, affordable, and efficient solution for sleep apnea management.

References:

[1] J. M. Parish and J. Shepard, "Clinical practice. Sleep apnea and CPAP therapy," *New England Journal of Medicine*, vol. 363, no. 2, pp. 106-114, Jul. 2010.

[2] S. G. Kandlikar, A. N. Kulkarni, and K. S. Bhavsar, "Advances in temperature sensing using thermistors," *Sensors and Actuators A: Physical*, vol. 154, no. 1, pp. 87-92, Jan. 2009.

[3] S. H. Kwon and H. K. Kim, "Design of a low-cost air pump for respiratory devices," *Journal of Biomedical Engineering*, vol. 45, no. 3, pp. 32-38, Mar. 2020.

[4] B. Smith and K. Lee, "Pressure monitoring and regulation in medical air devices," *IEEE Transactions on Biomedical Engineering*, vol. 66, no. 5, pp. 1214-1221, May 2019.

[5] C. H. Lin, "Application of OLED displays in portable health monitoring systems," *IEEE Transactions on Display Technology*, vol. 20, no. 3, pp. 224-230, Mar. 2014.

[6] P. Berry and R. Thomas, "Trends in sleep apnea treatments: From CPAP to advanced AI-based systems," *Journal of Sleep Research*, vol. 28, no. 4, pp. 10-17, Jul. 2019.

[7] L. K. Rao, "Development of affordable medical solutions for low-income countries," *IEEE Access*, vol. 8, pp. 19023-19031, Jan. 2020.

Code:

```
// include the library code:
#include "DHT.h"
#include <LiquidCrystal.h>

#include <Wire.h>
#include <Adafruit_Fingerprint.h>
#include <HardwareSerial.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

//#include <LiquidCrystal_I2C.h>
//LiquidCrystal_I2C lcd(0x27,16,2);

//----------------------------------------
#define SCREEN_WIDTH 128 // OLED display width, in pixels
#define SCREEN_HEIGHT 64 // OLED display height, in pixels
// Defines the DHT11 PIN and DHT sensor type.

//#define enA 10//Enable1 L298 Pin enA
#define in1 7 //Motor1  L298 Pin in1
#define in2 6 //Motor1  L298 Pin in1
#define in3 5 //Motor2  L298 Pin in1
#define in4 4 //Motor2  L298 Pin in1
//#define enB 5 //Enable2 L298 Pin enB

int Speed = 90;

int y1=8;
int y2=9;
int EN1=10;
int y4=11;

int lockpin = 2;


int enter1;
int exit1;
int counter;
```

```
int refval;

int enter2;
int exit2;
int refva2;

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

int tmpL = 7;
int tmpH = 13;
int rhL = 9;
int inPin = 10;
int rhData1 ;
int rhData2 ;
int rhValue = 0;
int ThermistorPin = A0;
int Vo;
float R1 = 10000;
float logR2, R2, T, Tc, Tf;
float c1 = 1.009249522e-03, c2 = 2.378405444e-04, c3 = 2.019202697e-07;

// initialize the library with the numbers of the interface pins

int pulsePin ;                 // Pulse Sensor purple wire connected to analog pin A0
int blinkPin ;                // pin to blink led at each beat
volatile int BPM;                   // int that holds raw Analog in 0. updated every 2mS
volatile int Signal;                // holds the incoming raw data
volatile int IBI = 600;             // int that holds the time interval between beats! Must be seeded!
volatile boolean Pulse = false;     // "True" when User's live heartbeat is detected. "False" when
not a "live beat".
volatile boolean QS = false;        // becomes true when Arduoino finds a beat.

static boolean serialVisual = true;   // Set to 'false' by Default.  Re-set to 'true' to see Arduino
Serial Monitor ASCII Visual Pulse

volatile int rate[10];                    // array to hold last ten IBI values
volatile unsigned long sampleCounter = 0;          // used to determine pulse timing
volatile unsigned long lastBeatTime = 0;           // used to find IBI
volatile int P = 0;                      // used to find peak in pulse wave, seeded
```

```
volatile int thresh = 525;               // used to find instant moment of heart beat, seeded
volatile int amp = 100;                  // used to hold amplitude of pulse waveform, seeded
volatile boolean firstBeat = true;       // used to seed rate array so we startup with reasonable
BPM
volatile boolean secondBeat = false;     // used to seed rate array so we startup with reasonable
BPM
//LiquidCrystal lcd(12, 11, 5, 4, 3, 2);


void setup() {
  Serial.begin(115200);
Serial.begin(9600);
if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { // Address 0x3D for 128x64
    Serial.println(F("SSD1306 allocation failed"));
    for(;;);
  }
  delay(20);
  display.clearDisplay();

  display.setTextSize(2);
  display.setTextColor(WHITE);
  display.setCursor(0, 10);
  // Display static text
  display.println("Starting");
  display.display();


  //setup pinmode
  pinMode(tmpL, OUTPUT);
  pinMode(tmpH, OUTPUT);
  pinMode(rhL, OUTPUT);
  pinMode(inPin, INPUT);

 pinMode(y1, OUTPUT);
 pinMode(y2, OUTPUT);
 pinMode(EN1, OUTPUT);

  digitalWrite(y1, LOW);
  digitalWrite(y2, LOW);
  analogWrite(EN1, 120);
```

```
}

void loop() {

  Vo = analogRead(ThermistorPin);
  R2 = R1 * (1023.0 / (float)Vo - 1.0);
  logR2 = log(R2);
  T = (1.0 / (c1 + c2 * logR2 + c3 * logR2 * logR2 * logR2));
  Tc = T - 273.15;
  Tf = (Tc * 9.0) / 5.0 + 32.0;


  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(WHITE);
  display.setCursor(0, 0);
  display.print("Temp(F) = ");
  display.setCursor(0, 30);
  display.print(Tf);
  display.display();

  Serial.println("Temperature(F)= ");
  Serial.println(Tf);



  if (Tf >= 77.00) {
  // When temperature is above or equal to 74.00°F
  digitalWrite(tmpH, HIGH);  // Activate heating element or indicator
  digitalWrite(y1, HIGH);    // Turn on control for y1
  digitalWrite(y2, LOW);     // Ensure y2 is off
  analogWrite(EN1, 100);     // Set PWM value for EN1 to 140

  // Display temperature and status on the OLED
  display.clearDisplay();
  display.setTextSize(2);
  display.setTextColor(WHITE);
  display.setCursor(0, 0);
  display.print("Temp(F) = ");
```

```
  display.setCursor(0, 15);
  display.print(Tf);
  display.setCursor(0, 40);
  display.print("Normal");
  display.display();
} else {
  // When temperature is below 74.00°F
  for (int pos = 0; pos <= 30; pos += 1) {
    // Read and calculate temperature from the thermistor
    Vo = analogRead(ThermistorPin);
    R2 = R1 * (1023.0 / (float)Vo - 1.0);
    logR2 = log(R2);
    T = (1.0 / (c1 + c2 * logR2 + c3 * logR2 * logR2 * logR2));
    Tc = T - 273.15;
    Tf = (Tc * 9.0) / 5.0 + 32.0;

    // Display temperature and status on the OLED
    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(WHITE);
    display.setCursor(0, 0);
    display.print("Temp(F) = ");
    display.setCursor(0, 15);
    display.print(Tf);
    display.setCursor(0, 40);
    display.print("Hampered");
    display.display();

    // Output values to Serial Monitor for debugging
    Serial.println("Temperature(F)= ");
    Serial.println(Tf);

    // Adjust air pump and other controls
    digitalWrite(y1, HIGH);
    digitalWrite(y2, LOW);
    analogWrite(EN1, 200);   // Increase PWM value to 240
    digitalWrite(tmpH, LOW); // Turn off heating element or indicator
    delay(500);              // Wait 500 milliseconds
  }
```

```
  for (int pos = 0; pos <= 30; pos += 1) {
    // Repeat temperature calculation and display updates
    Vo = analogRead(ThermistorPin);
    R2 = R1 * (1023.0 / (float)Vo - 1.0);
    logR2 = log(R2);
    T = (1.0 / (c1 + c2 * logR2 + c3 * logR2 * logR2 * logR2));
    Tc = T - 273.15;
    Tf = (Tc * 9.0) / 5.0 + 32.0;

    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(WHITE);
    display.setCursor(0, 0);
    display.print("Temp(F) = ");
    display.setCursor(0, 15);
    display.print(Tf);
    display.setCursor(0, 40);
    display.print("Hampered");

    display.display();

    Serial.println("Temperature(F) = ");
    Serial.println(Tf);

    digitalWrite(y1, HIGH);
    digitalWrite(y2, LOW);
    analogWrite(EN1, 100);   // Reduce PWM value to 140
    digitalWrite(tmpH, LOW); // Ensure heating element or indicator is off
    delay(500);              // Wait 500 milliseconds
  }
}

delay(600); // Wait 600 milliseconds before the next loop


}
```